CENTRALIZED EGRESS FIREWALL

PPT-STYLE SLIDE-BY-SLIDE EXPLANATION

4-member presentation guide

This document is written like speaker notes for a project PPT: each slide has what should be explained and what the presenter can say.

Implementation: EC2-based centralized firewall/router using Linux nftables, IP forwarding and NAT.

Region: eu-north-1 (Europe/Stockholm) **VPC:** 10.0.0.0/16

Important: Present this as an EC2-based firewall proof of concept, not as AWS Network Firewall.

Slide 1 — Title / Introduction

Presented by: Member 1

- Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall
- Main goal: control outbound Internet traffic from a private workload
- Private client does not have a public IPv4 address
- Traffic is centralized through a firewall before reaching the Internet

What to show on the PPT/screenshot: Use the project title and a simple client → firewall → Internet diagram.

WHAT TO SAY

Good morning everyone. Our project is titled Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall. The main goal of our project is to control outbound Internet traffic from a private AWS workload. Instead of allowing the client to directly access the Internet, we force its traffic through a centralized firewall. This gives us one controlled point where outbound traffic can be inspected and filtered.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 2 — Problem Statement

Presented by: Member 1

- Private workloads should not have direct public Internet exposure
- Uncontrolled outbound traffic can create security risk
- Organizations need centralized egress policies
- The project demonstrates this concept using a low-cost EC2-based design

What to show on the PPT/screenshot: Show a before/after concept: direct client Internet access versus controlled client → firewall → Internet.

WHAT TO SAY

The problem we are addressing is uncontrolled outbound access. In a real organization, many private workloads may need Internet access for updates, APIs or web services, but giving every workload direct Internet access makes centralized control difficult. Our solution is to create a single outbound path through a firewall, so security rules can be applied consistently.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 3 — Architecture

Presented by: Member 1

- VPC: 10.0.0.0/16
- Firewall subnet: 10.0.1.0/24
- Private client subnet: 10.0.2.0/24
- Firewall: 10.0.1.66
- Client: 10.0.2.171
- Traffic flow: Client → Firewall → Internet Gateway → Internet

What to show on the PPT/screenshot: Show the architecture diagram and the IP addresses.

WHAT TO SAY

Our architecture contains one VPC and two subnets. The firewall is placed in the firewall subnet and the client is placed in the private subnet. The client has private IP 10.0.2.171, while the firewall has private IP 10.0.1.66. The important point is that the client sends Internet-bound traffic to the firewall first. The firewall then forwards permitted traffic toward the Internet Gateway.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 4 — VPC, Subnets and Internet Gateway

Presented by: Member 2

- VPC CIDR: 10.0.0.0/16
- Firewall public subnet: 10.0.1.0/24
- Client private subnet: 10.0.2.0/24
- Internet Gateway: egress-igw
- Firewall subnet has the Internet route

What to show on the PPT/screenshot: Show the VPC, subnet and Internet Gateway console screenshots.

WHAT TO SAY

Now I will explain the AWS networking. We created a VPC with CIDR 10.0.0.0/16. Inside it, we created two subnets. The firewall subnet is 10.0.1.0/24 and the client subnet is 10.0.2.0/24. The Internet Gateway is attached to the VPC and provides the external Internet path for the firewall side.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 5 — Route Tables: The Key Control

Presented by: Member 2

- Firewall route table: 0.0.0.0/0 → Internet Gateway
- Client route table: 0.0.0.0/0 → firewall ENI
- Firewall ENI: eni-0f52bd38ad7c60d70
- Firewall private IP: 10.0.1.66
- This forces the client's Internet traffic through the firewall

What to show on the PPT/screenshot: Show the client-private-rt and firewall-public-rt screenshots.

WHAT TO SAY

The route tables are one of the most important parts of the project. The firewall subnet sends its default route to the Internet Gateway. But the client subnet does not send its default route directly to the Internet Gateway. Its default route points to the firewall ENI at 10.0.1.66. Therefore, the client is forced to use the centralized firewall for Internet-bound traffic.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 6 — Security Groups and Source/Destination Check

Presented by: Member 2

- Firewall security group controls management and web traffic
- Client security group allows required management traffic
- Firewall outbound traffic is permitted
- Source/destination checking was disabled on the firewall ENI
- Reason: the EC2 instance forwards packets belonging to another host

What to show on the PPT/screenshot: Show the firewall ENI source/destination check setting and security group rules.

WHAT TO SAY

We also configured security groups for the firewall and client. A critical setting was source/destination checking. Normally, an EC2 instance is expected to be the source or destination of traffic. Our firewall is different because it forwards traffic for the client. Therefore, source/destination checking had to be disabled on the firewall network interface.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 7 — Firewall EC2 and IP Forwarding

Presented by: Member 3

- EC2 instance name: central-firewall
- Private IP: 10.0.1.66
- Linux IP forwarding enabled
- Interface: ens5
- The EC2 instance acts as both router and firewall

What to show on the PPT/screenshot: Show the firewall EC2 console and the IP forwarding command/result.

WHAT TO SAY

Now I will explain the firewall implementation. We launched an Amazon Linux 2023 EC2 instance called central-firewall. Its private IP is 10.0.1.66. We enabled Linux IPv4 forwarding because the instance must forward packets received from the private client toward the Internet. This makes the EC2 instance act as a router as well as a firewall.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 8 — nftables Firewall Policy

Presented by: Member 3

- nftables is the Linux packet filtering framework used
- FORWARD chain policy is DROP
- Established/related traffic is accepted
- New TCP traffic from 10.0.2.0/24 to ports 80 and 443 is allowed
- Everything else is denied by the forwarding policy

What to show on the PPT/screenshot: Show the nft list ruleset / forward chain screenshot.

WHAT TO SAY

For filtering, we use nftables. The forwarding chain has a default DROP policy. This means traffic is denied unless it matches an allow rule. We allow established and related connections, and we allow new TCP connections from the client subnet to ports 80 and 443. Port 80 is HTTP and port 443 is HTTPS. This demonstrates a deny-by-default security model.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 9 — NAT / Masquerading

Presented by: Member 3

- Client source network: 10.0.2.0/24
- Outgoing interface: ens5
- POSTROUTING uses masquerade
- Private client address is translated for Internet communication
- Return traffic can be associated with the original connection

What to show on the PPT/screenshot: Show the nft postrouting / masquerade rule.

WHAT TO SAY

The client uses a private IP address, so we also configured source NAT using masquerading. In the POSTROUTING chain, traffic from 10.0.2.0/24 leaving through ens5 is masqueraded. This translates the private source address for outbound communication and allows responses from the Internet to return through the firewall.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 10 — Client Routing and Connectivity Test

Presented by: Member 4

- Client IP: 10.0.2.171
- Client has no public IP
- Default route is via AWS subnet gateway
- AWS subnet route sends Internet traffic toward firewall ENI
- Direct test to firewall private IP reached the firewall

What to show on the PPT/screenshot: Show the client ip route and curl-to-10.0.1.66 screenshots.

WHAT TO SAY

I will explain the testing. First, we verified that the client has private IP 10.0.2.171 and no public IP. The operating system shows the normal subnet gateway as its next hop, while the AWS route table controls the actual subnet-level path toward the firewall. We also tested connectivity to the firewall private address. The connection was refused immediately because there was no service listening on that port, which still demonstrated that the client could reach the firewall.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 11 — HTTP and HTTPS Testing

Presented by: Member 4

- HTTP request to example.com succeeded
- HTTPS connection to example.com succeeded
- Traffic originated from private client 10.0.2.171
- Success proves the complete egress path works
- Filtering rules allow the required web traffic

What to show on the PPT/screenshot: Show the HTTP curl and HTTPS curl screenshots.

WHAT TO SAY

Next, we tested Internet access from the private client. The HTTP curl request to example.com returned a successful response. We also tested HTTPS and successfully established the TLS connection and received an HTTP response. These tests prove that the private client can reach the Internet through the centralized firewall and that our HTTP and HTTPS forwarding rules are working.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 12 — Systems Manager / Management Validation

Presented by: Member 4

- Client appeared as an online managed node
- SSM agent was active and running
- Session Manager terminal was successfully opened
- Client did not require a public IP for management
- This helped us operate and test the private instance

What to show on the PPT/screenshot: Show the Fleet Manager online node and Session Manager screenshots.

WHAT TO SAY

We also validated management connectivity using AWS Systems Manager. The egress-client appeared as an online managed node, and we successfully opened a Session Manager terminal. The SSM agent was active and had valid IAM role credentials. This allowed us to manage and test the private client without giving it a public IP.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 13 — Final Validation

Presented by: Member 4

- nftables final ruleset verified
- /etc/nftables/main.nft contains the intended policy
- nft syntax check completed successfully
- HTTP/HTTPS tests succeeded
- Centralized egress path is demonstrated

What to show on the PPT/screenshot: Show the final nft ruleset and nftables syntax-check screenshot.

WHAT TO SAY

Finally, we verified the final nftables ruleset and the saved configuration file. The syntax check completed successfully. Together with the successful HTTP and HTTPS tests, this confirms that our centralized egress path is functioning as designed.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 14 — Conclusion

Presented by: All Members / Member 4

- Private workload has no public IP
- Internet traffic is centralized through the firewall
- nftables applies deny-by-default forwarding
- HTTP/HTTPS are explicitly permitted
- NAT enables private-client Internet access
- Project demonstrates the core centralized egress concept

What to show on the PPT/screenshot: End with the architecture diagram and a final 'Client → Firewall → Internet' flow.

WHAT TO SAY

To conclude, our project successfully demonstrates centralized outbound traffic control. The private client does not have a public IP and its Internet-bound traffic is routed through the EC2 firewall. nftables provides deny-by-default filtering, HTTP and HTTPS are explicitly allowed, and source NAT enables Internet access. This demonstrates the core concept of centralized egress filtering in AWS using a low-cost EC2-based proof of concept.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Slide 15 — Viva: Questions You Should Know

Presented by: All Members

- What is egress traffic? — Traffic leaving the AWS environment
- Why is the client private? — To avoid direct public exposure
- Why use nftables? — Linux packet filtering and NAT
- Why DROP policy? — Deny by default
- Why ports 80/443? — HTTP/HTTPS
- Why masquerading? — Source NAT for private Internet access
- Why disable source/destination check? — Firewall forwards traffic for another host
- Is this AWS Network Firewall? — No; EC2-based proof of concept

What to show on the PPT/screenshot: Use this as the final revision slide before the actual presentation.

WHAT TO SAY

Each member should understand the complete architecture even if they present only one section. If a question crosses sections, explain the relationship between routing, firewall filtering, NAT and testing.

Project: Centralized Egress Traffic Filtering Using AWS VPC and an EC2-Based Firewall

Final Presentation Tip

Do not read every bullet. Point to the architecture or screenshot and explain the reason behind each configuration. The most important sentence to remember is:

“Our private client sends Internet-bound traffic through a centralized EC2 firewall, where nftables applies a deny-by-default policy and NAT enables controlled Internet access.”

Recommended order: Member 1 → Member 2 → Member 3 → Member 4 → Questions.